Due Date: 4/14/2026 Names: Aiden Wang

CSCI 240 PA6 Submission

Exercise 1

Source code below:

```
package PA6;
import net.datastructures.Entry;
import net.datastructures.SortedPriorityQueue;
import java.util.Comparator;
public class PA6_Ex1 {
   public static class MinComparator implements Comparator<Integer> {
       @Override
       public int compare(Integer a, Integer b) {
           return a.compareTo(b);
       }
   }
   public static class MaxComparator implements Comparator<Integer> {
       @Override
       public int compare(Integer a, Integer b) {
           return b.compareTo(a); // Reverse comparison order
       }
   }
   public static void main(String[] args) {
       System.out.println("Modified by: Aiden Wang\n");

       SortedPriorityQueue<Integer, String> minPQ = new
SortedPriorityQueue<>(new MinComparator());
       SortedPriorityQueue<Integer, String> maxPQ = new
SortedPriorityQueue<>(new MaxComparator());

       System.out.println("=== Testing Min-Priority Queue (Lowest value
has highest priority) ===");
       testPQ(minPQ);

       System.out.println("\n=== Testing Max-Priority Queue (Largest
value has highest priority) ===");
       testPQ(maxPQ);
   }
   public static void testPQ(SortedPriorityQueue<Integer, String> pq) {
       // insert(5, “five”), insert(4, “four”), insert(7, “seven”),
insert(1, “one”)
       pq.insert(5, "five");
       pq.insert(4, "four");
       pq.insert(7, "seven");
       pq.insert(1, "one");
     // min(), removeMin()
       printMin(pq);
       pq.removeMin();
       // insert(3, “three”), insert(6, “six”)
       pq.insert(3, "three");
       pq.insert(6, "six");
    // min(), removeMin(), min(), removeMin()
       printMin(pq);
       pq.removeMin();
       printMin(pq);
       pq.removeMin();
```

```
        // insert(8, “eight”)
        pq.insert(8, "eight");
 // min(), removeMin()
        printMin(pq);
        pq.removeMin();
        // insert(2, “two”)
        pq.insert(2, "two");
        // min(), removeMin(), min(), removeMin()
        printMin(pq);
        pq.removeMin();
        printMin(pq);
        pq.removeMin();
    }

    private static void printMin(SortedPriorityQueue<Integer, String> pq)
{
        Entry<Integer, String> minEntry = pq.min();
        if (minEntry != null) {
            System.out
                    .println("min() output -> Key: " + minEntry.getKey() +
", Value: \"" + minEntry.getValue() + "\"");
        }
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/
Modified by: Aiden Wang

=== Testing Min-Priority Queue (Lowest value has highest priority) ===
min() output -> Key: 1, Value: "one"
min() output -> Key: 3, Value: "three"
min() output -> Key: 4, Value: "four"
min() output -> Key: 5, Value: "five"
min() output -> Key: 2, Value: "two"
min() output -> Key: 6, Value: "six"

=== Testing Max-Priority Queue (Largest value has highest priority) ===
min() output -> Key: 7, Value: "seven"
min() output -> Key: 6, Value: "six"
min() output -> Key: 5, Value: "five"
min() output -> Key: 8, Value: "eight"
min() output -> Key: 4, Value: "four"
min() output -> Key: 3, Value: "three"

Process finished with exit code 0
```

Due Date: 4/14/2026 Names: Aiden Wang

Exercise 2
Source code below:

```
package PA6;
import net.datastructures.Entry;
import net.datastructures.SortedPriorityQueue;
import java.io.File;
import java.io.FileNotFoundException;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;
public class PA6_Ex2 {
   public static class MaxComparator implements Comparator<Integer> {
       @Override
       public int compare(Integer a, Integer b) {
           return b.compareTo(a); // Reverse comparison order for
Max-Priority Queue
       }
   }
   public static void main(String[] args) {
       System.out.println("Modified by: Aiden Wang\n");
       System.out.println("=== Sorting small1k.txt using
SortedPriorityQueue ===");
       sortAndMeasure("PAs/PA6/data/small1k.txt", 1000);

       System.out.println("\n=== Sorting large100k.txt using
SortedPriorityQueue ===");
       System.out.println("Warning: O(n^2) complexity with 100k elements
can take several seconds to a minute...");
       sortAndMeasure("PAs/PA6/data/large100k.txt", 100000);
   }

   public static void sortAndMeasure(String filename, int expectedSize) {
       List<Integer> data = new ArrayList<>(expectedSize);

       try (Scanner scanner = new Scanner(new File(filename))) {
           while (scanner.hasNextInt()) {
               data.add(scanner.nextInt());
           }
       } catch (FileNotFoundException e) {
           System.err.println("Could not find file: " + filename);
           System.err.println("Make sure to run the program from the
project root.");
           return;
       }
       SortedPriorityQueue<Integer, String> maxPQ = new
SortedPriorityQueue<>(new MaxComparator());
       List<Integer> sortedData = new ArrayList<>(data.size());

       // Start timing
       long startTime = System.currentTimeMillis();

       // Phase 1: Insert all elements into max PQ
       for (Integer val : data) {
           maxPQ.insert(val, String.valueOf(val));
       }
```

```
        while (!maxPQ.isEmpty()) {
            Entry<Integer, String> entry = maxPQ.removeMin();
            sortedData.add(entry.getKey());
        }
        // End timing
        long endTime = System.currentTimeMillis();
        long elapsedMs = endTime - startTime;

        // Output formatting: first 5 and last 5 elements
        System.out.print("First 5 values: ");
        for (int i = 0; i < Math.min(5, sortedData.size()); i++) {
            System.out.print(sortedData.get(i) + " ");
        }
        System.out.println();

        System.out.print("Last 5 values:  ");
        for (int i = Math.max(0, sortedData.size() - 5); i <
sortedData.size(); i++) {
            System.out.print(sortedData.get(i) + " ");
        }
        System.out.println();

        System.out.println("-> Total sorting time: " + elapsedMs + " ms");
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java
Modified by: Aiden Wang

=== Sorting small1k.txt using SortedPriorityQueue ===
First 5 values: 8191 8183 8175 8167 8163
Last 5 values:  59 39 15 11 7
-> Total sorting time: 9 ms

=== Sorting large100k.txt using SortedPriorityQueue ===
Warning: O(n^2) complexity with 100k elements can take several seconds to a minute...
First 5 values: 100000 99999 99998 99997 99996
Last 5 values:  5 4 3 2 1
-> Total sorting time: 35911 ms

Process finished with exit code 0
```

Exercise 3
Source code below:

Due Date: 4/14/2026 Names: Aiden Wang

```
package PA6;
import net.datastructures.Entry;
import net.datastructures.HeapPriorityQueue;
import java.util.Comparator;
public class PA6_Ex3 {
   public static class MinComparator implements Comparator<Integer> {
       @Override
       public int compare(Integer a, Integer b) {
           return a.compareTo(b);
       }
   }
   public static class MaxComparator implements Comparator<Integer> {
       @Override
       public int compare(Integer a, Integer b) {
           return b.compareTo(a); // Reverse comparison order
       }
   }
   public static void main(String[] args) {
       System.out.println("Modified by: Aiden Wang\n");
       HeapPriorityQueue<Integer, String> minPQ = new
HeapPriorityQueue<>(new MinComparator());
       HeapPriorityQueue<Integer, String> maxPQ = new
HeapPriorityQueue<>(new MaxComparator());

       System.out.println("=== Testing Min-Priority Queue
(HeapPriorityQueue) ===");
       testPQ(minPQ);

       System.out.println("\n=== Testing Max-Priority Queue
(HeapPriorityQueue) ===");
       testPQ(maxPQ);
   }

   public static void testPQ(HeapPriorityQueue<Integer, String> pq) {

       pq.insert(5, "five");
       pq.insert(4, "four");
       pq.insert(7, "seven");
       pq.insert(1, "one");

       printMin(pq);
       pq.removeMin();

       pq.insert(3, "three");
       pq.insert(6, "six");
)
       printMin(pq);
       pq.removeMin();
       printMin(pq);
       pq.removeMin();

       pq.insert(8, "eight");

       printMin(pq);
       pq.removeMin();

       pq.insert(2, "two");
```

```
        printMin(pq);
        pq.removeMin();
        printMin(pq);
        pq.removeMin();
    }

    private static void printMin(HeapPriorityQueue<Integer, String> pq) {
        Entry<Integer, String> minEntry = pq.min();
        if (minEntry != null) {
            System.out.println("min() output -> Key: " + minEntry.getKey()
+ ", Value: \"" + minEntry.getValue() + "\"");
        }
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java
Modified by: Aiden Wang

=== Sorting small1k.txt using SortedPriorityQueue ===
First 5 values: 8191 8183 8175 8167 8163
Last 5 values:  59 39 15 11 7
-> Total sorting time: 9 ms

=== Sorting large100k.txt using SortedPriorityQueue ===
Warning: O(n^2) complexity with 100k elements can take several seconds to a minute...
First 5 values: 100000 99999 99998 99997 99996
Last 5 values:  5 4 3 2 1
-> Total sorting time: 35911 ms

Process finished with exit code 0
```

Exercise 4

Source code below:

```
package PAs.PA6;
import net.datastructures.Entry;
```

Due Date: 4/14/2026 Names: Aiden Wang

```java
import net.datastructures.HeapPriorityQueue;
import java.io.File;
import java.io.FileNotFoundException;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;
public class PA6_Ex4 {
   public static class MaxComparator implements Comparator<Integer> {
       @Override
       public int compare(Integer a, Integer b) {
           return b.compareTo(a); // Reverse comparison order for
Max-Priority Queue
       }
   }
   public static void main(String[] args) {
       System.out.println("Modified by: Aiden Wang\n");

       System.out.println("=== Sorting small1k.txt using
HeapPriorityQueue ===");
       sortAndMeasure("PAs/PA6/data/small1k.txt", 1000);

       System.out.println("\n=== Sorting large100k.txt using
HeapPriorityQueue ===");
       System.out.println("Notice how much faster this O(n log n)
algorithm runs compared to Ex2!");
       sortAndMeasure("PAs/PA6/data/large100k.txt", 100000);
   }
   public static void sortAndMeasure(String filename, int expectedSize){
       List<Integer> data = new ArrayList<>(expectedSize);
       try (Scanner scanner = new Scanner(new File(filename))) {
           while (scanner.hasNextInt()) {
               data.add(scanner.nextInt());
           }
       } catch (FileNotFoundException e) {
           System.err.println("Could not find file: " + filename);
           System.err.println("Make sure to run the program from the
project root.");
           return;
       }
       HeapPriorityQueue<Integer, String> maxPQ = new
HeapPriorityQueue<>(new MaxComparator());
       List<Integer> sortedData = new ArrayList<>(data.size());
       long startTime = System.currentTimeMillis();
       for (Integer val : data) {
           maxPQ.insert(val, String.valueOf(val));
       }
       while (!maxPQ.isEmpty()) {
           Entry<Integer, String> entry = maxPQ.removeMin();
           sortedData.add(entry.getKey());
       }
       long endTime = System.currentTimeMillis();
       long elapsedMs = endTime - startTime;
       // Output formatting: first 5 and last 5 elements
       System.out.print("First 5 values: ");
       for (int i = 0; i < Math.min(5, sortedData.size()); i++) {
           System.out.print(sortedData.get(i) + " ");
```

```
        }
        System.out.println();
        System.out.print("Last 5 values:  ");
        for (int i = Math.max(0, sortedData.size() - 5); i <
sortedData.size(); i++) {
            System.out.print(sortedData.get(i) + " ");
        }
        System.out.println();
        System.out.println("-> Total sorting time: " + elapsedMs + " ms");
    }
}
```

Input/output below:

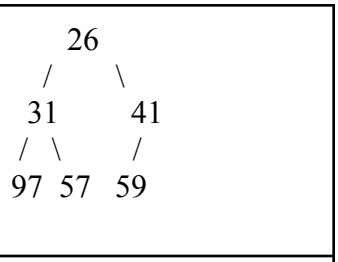
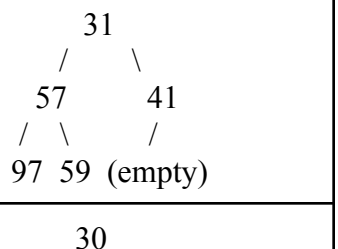
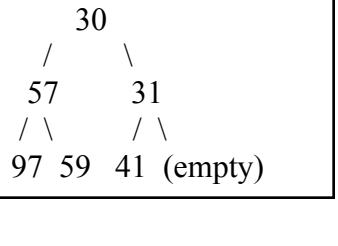
```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/java -java
Modified by: Aiden Wang

=== Sorting small1k.txt using HeapPriorityQueue ===
First 5 values: 8191 8183 8175 8167 8163
Last 5 values:  59 39 15 11 7
-> Total sorting time: 4 ms

=== Sorting large100k.txt using HeapPriorityQueue ===
Notice how much faster this O(n log n) algorithm runs compared to Ex2!
First 5 values: 100000 99999 99998 99997 99996
Last 5 values:  5 4 3 2 1
-> Total sorting time: 111 ms

Process finished with exit code 0
```

Answer for Question 1

A Priority Queue (PQ) is an abstract data type that stores a collection of entries (key-value pairs) where each entry has an associated priority. Unlike a standard queue that operates strictly on a First-In, First-Out (FIFO) basis, a priority queue removes elements based on their priority—specifically, the element with the lowest key (highest priority) is removed first using the removeMin() operation. How to use it as a regular FIFO queue: You can simulate a standard FIFO queue using a priority queue by assigning an incrementing integer (such as a counter or timestamp) as the key for each newly inserted element. The first item inserted gets key 1, the second gets key 2, and so on. Because a min-oriented priority queue removes the item with the smallest key first, it will successfully remove the oldest inserted elements first, perfectly mimicking FIFO behavior.

Answer for Question 2

Due Date: 4/14/2026 Names: Aiden Wang

Initial Heap	
After removeMin()	
After insert(30)	

Extra Credit – Option 1

Source code below:

```
package PA6;

import net.datastructures.Entry;
import net.datastructures.HeapPriorityQueue;

import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Scanner;

public class PA6_EC1 {

   // Comparator that compares strings in descending order
   public static class StringMaxComparator implements Comparator<String>
{
       @Override
       public int compare(String a, String b) {
           return b.compareTo(a); // Reverse alphabetical/string
comparison order
       }
   }
```

```
   public static void main(String[] args) {
       System.out.println("Modified by: Aiden Wang\n");
       System.out.println("=== Running Extra Credit Option 1 ===");

       String inputFile = "PAs/PA6/data/large100k.txt";
       String outputFile = "PAs/PA6/data/large100k_EC_output.txt";

       sortStringsToOutputFile(inputFile, outputFile, 100000);
   }

   public static void sortStringsToOutputFile(String inputPath, String
outputPath, int expectedSize) {
       List<Integer> data = new ArrayList<>(expectedSize);

       // 1. Read all integer data in first
       try (Scanner scanner = new Scanner(new File(inputPath))) {
           while (scanner.hasNextInt()) {
               data.add(scanner.nextInt());
           }
       } catch (FileNotFoundException e) {
           System.err.println("Could not find input file: " + inputPath);
           return;
       }

       System.out.println("Data loaded. Inserting into
HeapPriorityQueue...");

       // 2. PQ uses <String, Integer> and a string descending comparator
       HeapPriorityQueue<String, Integer> maxStringPQ = new
HeapPriorityQueue<>(new StringMaxComparator());

       // Insert <String, Integer> entries
       for (Integer val : data) {
           maxStringPQ.insert(String.valueOf(val), val);
       }

       System.out.println("Sorting and writing to output file...");

       // 3. Extract and write to file format exactly as requested: 5
values per line
       try (PrintWriter writer = new PrintWriter(new File(outputPath))) {
           int count = 0;

           while (!maxStringPQ.isEmpty()) {
               Entry<String, Integer> entry = maxStringPQ.removeMin();
               writer.print(entry.getKey());
               count++;

               // Format: 5 items per line space-separated
               if (count % 5 == 0 || maxStringPQ.isEmpty()) {
                   writer.println();
               } else {
                   writer.print(" ");
               }
           }
           System.out.println("\nSUCCESS: Output successfully written to
-> " + outputPath);
```

```
            System.out.println("Please open this file and take your
screenshot for submission!");

        } catch (FileNotFoundException e) {
            System.err.println("Failed to write to output file: " +
e.getMessage());
        }
    }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22.0.2/Contents/Home/bin/
Modified by: Aiden Wang

=== Running Extra Credit Option 1 ===
Data loaded. Inserting into HeapPriorityQueue...
Sorting and writing to output file...

SUCCESS: Output successfully written to -> PAs/PA6/data/large100k_EC_output.txt
Please open this file and take your screenshot for submission!

Process finished with exit code 0
```